Full Stack Web Development

Complete Guide with Detailed Explanations, Code and Practice Problems

---

Stack covered: HTML, CSS, JavaScript, React, Java, Spring Boot, SQL/JPA, JWT Security, Testing, Git, Docker and Deployment

Contents

1. How the Web Works (HTTP, REST, architecture)
2. HTML in Depth
3. CSS in Depth (Box Model, Flexbox, Grid, Responsive)
4. JavaScript (Core, ES6+, DOM, Async) + 10 problems
5. React (Hooks, Forms, Router, Context) + 5 problems
6. Java Essentials for Backend + 5 problems
7. Spring Boot REST API (complete Student CRUD project)
8. SQL, Databases and JPA + 8 problems
9. Authentication and Security (BCrypt, JWT)
10. Testing (JUnit, Mockito, MockMvc)
11. Git and GitHub
12. Docker and Deployment
13. Capstone Project Blueprint
14. Common Errors and Fixes
15. Interview Questions and Answers
16. 8-Week Study Plan

1. How the Web Works

Before writing code you must understand what happens when a browser talks to a server. Every full stack application is built on this one idea: a **client** sends a **request**, and a **server** sends back a **response**.

1.1 Architecture of a typical full stack app

```
Browser (React app: HTML + CSS + JS)
      |   HTTP request  (JSON)          ^  HTTP response (JSON)
      v                                 |
Spring Boot server
   Controller  ->  Service  ->  Repository (JPA/Hibernate)
      |   JDBC / SQL                    ^  rows
      v                                 |
Database (MySQL / PostgreSQL)
```

- **Frontend**: what the user sees and clicks (React, HTML, CSS, JS).
- **Backend**: business rules, validation, security, talks to the database (Spring Boot).
- **Database**: permanent storage (MySQL, PostgreSQL, MongoDB).

1.2 HTTP request and response

A request has a **method**, a **URL**, **headers** and an optional **body**. A response has a **status code**, **headers** and a **body**.

```
POST /api/students HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{ "name": "Asha", "email": "asha@mail.com", "age": 21 }

----- response -----
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 7, "name": "Asha", "email": "asha@mail.com", "age": 21 }
```

1.3 HTTP methods

Method	Purpose	Has body?	Idempotent?
GET	Read data	No	Yes
POST	Create a new resource	Yes	No
PUT	Replace a whole resource	Yes	Yes
PATCH	Update part of a resource	Yes	Usually no
DELETE	Remove a resource	No	Yes

Idempotent means calling it many times has the same effect as calling it once. Deleting student 7 twice still leaves student 7 deleted; posting twice creates two students.

1.4 Status codes you must know

Code	Meaning	When to use
200 OK	Success	GET/PUT success
201 Created	Resource created	POST success
204 No Content	Success, nothing to return	DELETE success
400 Bad Request	Invalid input	Validation failed

Code	Meaning	When to use
401 Unauthorized	Not logged in / bad token	Missing or invalid JWT
403 Forbidden	Logged in but not allowed	User lacks the role
404 Not Found	Resource does not exist	ID not in database
409 Conflict	Duplicate / state conflict	Email already registered
500 Server Error	Bug on the server	Unhandled exception

1.5 Anatomy of a URL

```
https://api.myapp.com:443/api/students?page=2&size=10#top
 |       |            |   |            |              |
scheme   host         port path        query string   fragment
```

- **Path parameter**: `/api/students/7` identifies one resource.
- **Query parameter**: `?page=2&size=10` filters, sorts or paginates a list.

1.6 What happens when you type a URL and press Enter?

- The browser checks its cache, then asks **DNS** to convert the domain name to an IP address.
- It opens a TCP connection, and for HTTPS does a **TLS handshake** to encrypt the traffic.
- It sends the HTTP request; the server processes it (maybe querying a database) and sends a response.
- The browser parses the HTML, downloads CSS/JS/images, builds the DOM, applies styles and runs JavaScript.

1.7 REST API design rules

- Use **nouns** for resources, not verbs: `/students` not `/getStudents`.
- Use plural names, and let the HTTP method express the action.
- Be **stateless**: each request carries everything the server needs (e.g. the token).
- Return correct status codes and a consistent JSON error shape.

Action	Request	Success status
List students	GET /api/students	200
Get one	GET /api/students/7	200 / 404
Create	POST /api/students	201
Update	PUT /api/students/7	200
Delete	DELETE /api/students/7	204

PROBLEM 1: Choose method and status

For each scenario give the HTTP method, URL and success status: (a) fetch all products on page 3, (b) user registers, (c) change only the price of product 5, (d) remove product 5.

Solution

```
(a) GET    /api/products?page=3&size=10      -> 200 OK
(b) POST   /api/users/register               -> 201 Created
(c) PATCH  /api/products/5   {"price": 499}  -> 200 OK
(d) DELETE /api/products/5                   -> 204 No Content
```

Explanation: (c) uses PATCH because only one field changes; PUT would require sending the whole product.

2. HTML in Depth

HTML (HyperText Markup Language) defines the **structure and meaning** of a web page. CSS decorates it and JavaScript makes it interactive.

2.1 Page skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My First Page</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header><h1>My Site</h1></header>
  <main>
    <p>Hello <strong>world</strong>!</p>
  </main>
  <script src="app.js" defer></script>
</body>
</html>
```

- The `viewport` meta tag is required for responsive design on phones.
- Use `defer` on scripts so they run after the HTML is parsed.

2.2 Important elements

Group	Tags
Text	`h1-h6 p span strong em br hr blockquote code pre`
Lists	`ul ol li` (unordered/ordered), `dl dt dd`
Links and media	`a href="..."`, `img src alt`, `video audio`
Semantic layout	`header nav main section article aside footer`
Tables	`table thead tbody tr th td`
Forms	`form label input select textarea button fieldset`

2.3 Block vs inline

Block elements (`div p h1 ul`) start on a new line and take the full width. **Inline** elements (`span a strong img`) flow within text and only take the width they need. Prefer **semantic** tags (`nav, article`) over generic `div` because they help SEO and screen readers.

2.4 Forms and input types

```
<form action="/register" method="post">
  <label for="name">Name</label>
  <input type="text" id="name" name="name" required minlength="3" />

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required />

  <label for="pwd">Password</label>
  <input type="password" id="pwd" name="password" required minlength="8" />

  <label>Gender</label>
  <input type="radio" name="gender" value="F" /> Female
  <input type="radio" name="gender" value="M" /> Male

  <select name="course">
    <option value="">--Select--</option>
```

```
    <option value="java">Java</option>
    <option value="web">Web</option>
  </select>

  <input type="checkbox" name="terms" required /> I agree
  <button type="submit">Register</button>
</form>
```

The **name** attribute is the key sent to the server; the **for/id** pair connects a label to its input so clicking the label focuses the field.

2.5 Images and why they sometimes do not show

- Always give `alt` text: `<img src="images/pic.jpg" alt="Profile photo">`.
- Paths are **case-sensitive on GitHub Pages and Linux servers**: `Pic.JPG` is not `pic.jpg`.
- Use **relative paths** (`images/pic.jpg`), not `C:/Users/...`, and commit the images folder to Git.
- Folder names with spaces or a leading slash (`/images/pic.jpg`) can break on project pages.

PROBLEM 2: Student profile card page

Create a page with a header, a navigation bar of three links, a profile image, and a table of 3 subjects with marks, using semantic tags.

Solution

```
<body>
  <header>
    <nav>
      <a href="#about">About</a> | <a href="#marks">Marks</a> | <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h1>Asha Kumar</h1>
      <img src="images/asha.jpg" alt="Photo of Asha" width="150" />
    </section>
    <section id="marks">
      <h2>Marks</h2>
      <table border="1">
        <thead><tr><th>Subject</th><th>Marks</th></tr></thead>
        <tbody>
          <tr><td>Java</td><td>88</td></tr>
          <tr><td>DBMS</td><td>79</td></tr>
          <tr><td>Web</td><td>91</td></tr>
        </tbody>
      </table>
    </section>
  </main>
  <footer id="contact">asha@mail.com</footer>
</body>
```

Explanation: Each `section` has an id so the nav links (`#about`) scroll to it.

3. CSS in Depth

3.1 Selectors and specificity

```
p { color: #333; }                 /* element selector     */
.card { padding: 16px; }           /* class selector       */
#header { background: navy; }      /* id selector          */
nav a:hover { color: tomato; }     /* descendant + pseudo  */
ul > li { list-style: none; }      /* direct child         */
input[type="email"] { border: 1px solid gray; }
```

Specificity decides which rule wins: inline style > id > class/attribute/pseudo-class > element. If equal, the rule that appears later wins. Avoid `!important`.

3.2 The box model

Every element is a box: **content + padding + border + margin**. Always set `box-sizing: border-box` so that width includes padding and border.

```
*, *::before, *::after { box-sizing: border-box; }
.card {
  width: 300px;
  padding: 16px;              /* space inside the border  */
  border: 1px solid #ccc;
  margin: 12px auto;          /* space outside; auto centers a block */
  border-radius: 8px;
}
```

3.3 Units and variables

- `px` fixed; `rem` relative to root font size (best for fonts/spacing); `%` relative to parent; `vw/vh` viewport width/height.
- CSS variables keep your theme in one place.

```
:root { --primary: #1f4e79; --radius: 8px; }
.btn { background: var(--primary); border-radius: var(--radius); color: #fff; padding: .5rem
      1rem; }
```

3.4 Positioning

Value	Behaviour
static	Default, normal flow
relative	Offset from its normal position; also a reference for absolute children
absolute	Removed from flow; positioned against nearest positioned ancestor
fixed	Stays at a position in the viewport while scrolling (sticky headers)
sticky	Acts relative until a scroll threshold, then sticks

3.5 Flexbox (one-dimensional layout)

```
.row {
  display: flex;
  justify-content: space-between;  /* main axis (horizontal by default) */
  align-items: center;             /* cross axis (vertical)             */
  gap: 16px;
  flex-wrap: wrap;
}
.row .item { flex: 1; }            /* share space equally */
```

3.6 Grid (two-dimensional layout)

```
.grid {
  display: grid;
```

```
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

This single rule creates a responsive card grid that automatically changes the number of columns.

3.7 Media queries (responsive design)

```
.container { width: 90%; max-width: 1100px; margin: 0 auto; }
.sidebar { display: none; }
@media (min-width: 768px) {          /* mobile first: add features for bigger screens */
  .sidebar { display: block; }
  .layout { display: grid; grid-template-columns: 250px 1fr; }
}
```

PROBLEM 3: Center a div both ways

Center a box horizontally and vertically in the middle of the screen.

Solution

```
body { margin: 0; min-height: 100vh; display: grid; place-items: center; }
/* or with flexbox */
body { margin: 0; min-height: 100vh; display: flex; justify-content: center; align-items: center;
      }
```

Explanation: Both methods center the child on both axes. `100vh` makes the body as tall as the screen.

PROBLEM 4: Responsive navbar

Build a navbar with the logo on the left and links on the right, that stacks vertically below 600px.

Solution

```
<nav class="nav">
  <div class="logo">MySite</div>
  <ul class="links"><li><a href="#">Home</a></li><li><a href="#">About</a></li></ul>
</nav>

.nav { display: flex; justify-content: space-between; align-items: center;
       padding: 12px 24px; background: #1f4e79; color: #fff; }
.links { display: flex; gap: 20px; list-style: none; margin: 0; padding: 0; }
.links a { color: #fff; text-decoration: none; }
@media (max-width: 600px) {
  .nav { flex-direction: column; gap: 8px; }
  .links { flex-direction: column; text-align: center; }
}
```

4. JavaScript (Core, ES6+, DOM, Async)

4.1 Variables and data types

```
const PI = 3.14;            // cannot be reassigned (use by default)
let count = 0;              // can be reassigned, block scoped
var old = 1;                // function scoped - avoid

// primitives: string, number, boolean, null, undefined, bigint, symbol
// reference types: object, array, function
typeof "hi";    // "string"
typeof null;    // "object"  (famous quirk)
0.1 + 0.2;      // 0.30000000000000004  (floating point)

// == converts types, === does not. Always use ===
"5" == 5;   // true
"5" === 5;  // false
// falsy values: false, 0, "", null, undefined, NaN
```

4.2 Functions

```
function add(a, b = 0) { return a + b; }       // default parameter
const mul = (a, b) => a * b;                   // arrow function
const greet = name => `Hello, ${name}!`;       // template literal
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);   // rest parameter
sum(1, 2, 3);   // 6
```

4.3 Arrays and objects

```
const nums = [1, 2, 3, 4, 5];
nums.map(n => n * 2);              // [2,4,6,8,10]   transform
nums.filter(n => n % 2 === 0);     // [2,4]          keep matching
nums.reduce((acc, n) => acc + n, 0);   // 15         combine to one value
nums.find(n => n > 3);             // 4
nums.some(n => n > 4);             // true
nums.every(n => n > 0);            // true
[...nums, 6];                      // spread: copy and add
[3, 1, 2].sort((a, b) => a - b);   // [1,2,3]  (default sort is alphabetical!)

const user = { name: "Asha", age: 21, address: { city: "Pune" } };
const { name, age } = user;                 // destructuring
const copy = { ...user, age: 22 };          // spread: shallow copy with override
user.address?.city;                         // optional chaining
user.phone ?? "N/A";                        // nullish coalescing
Object.keys(user); Object.entries(user);    // keys / [key, value] pairs
JSON.stringify(user); JSON.parse('{"a":1}');   // object <-> JSON text
```

4.4 Scope, closures and this

A **closure** is a function that remembers variables from the place where it was created, even after that outer function has finished.

```
function makeCounter() {
  let count = 0;                    // private variable
  return () => ++count;
}
const c = makeCounter();
c(); c(); c();      // 1, 2, 3  -> count survives between calls
```

this: in a normal function it depends on *how it is called*; in an **arrow function** it is taken from the surrounding scope. That is why arrow functions are preferred for callbacks.

4.5 DOM manipulation and events

```
<ul id="list"></ul>
<input id="item" /> <button id="add">Add</button>

<script>
  const input = document.querySelector("#item");
  const list = document.querySelector("#list");
  document.querySelector("#add").addEventListener("click", () => {
```

```
    if (!input.value.trim()) return;
    const li = document.createElement("li");
    li.textContent = input.value;           // textContent is safe from XSS
    list.appendChild(li);
    input.value = "";
  });
  // Event delegation: one listener on parent handles all children
  list.addEventListener("click", e => { if (e.target.tagName === "LI") e.target.remove(); });
</script>
```

4.6 Asynchronous JavaScript

JavaScript is single threaded. Slow tasks (network, timers) are handed to the browser and their callbacks wait in a queue. The **event loop** runs them when the call stack is empty. **Promise callbacks (microtasks) run before timer callbacks (macrotasks).**

```
async function loadUsers() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("HTTP " + res.status);   // fetch does NOT throw on 404/500
    return await res.json();
  } catch (err) {
    console.error("Failed:", err.message);
  }
}

// Run in parallel
const [users, posts] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/posts").then(r => r.json()),
]);

// POST with JSON
await fetch("/api/students", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Asha", email: "a@b.com", age: 21 }),
});
```

4.7 Practice problems (with solutions)

PROBLEM 5: Reverse a string and check palindrome

Write `reverse(s)` and `isPalindrome(s)` (ignore case, spaces and punctuation).

Solution

```
const reverse = s => [...s].reverse().join("");

function isPalindrome(s) {
  const clean = s.toLowerCase().replace(/[^a-z0-9]/g, "");
  let i = 0, j = clean.length - 1;
  while (i < j) if (clean[i++] !== clean[j--]) return false;
  return true;
}
isPalindrome("A man, a plan, a canal: Panama");   // true
```

Explanation: Two pointers compare from both ends: O(n) time, no extra reversed copy needed.

PROBLEM 6: Two Sum

Given an array and a target, return the indices of two numbers that add up to the target.

Solution

```
function twoSum(nums, target) {
  const seen = new Map();                 // value -> index
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);
  }
  return [];
```

```
}
twoSum([2, 7, 11, 15], 9);   // [0, 1]
```

Explanation: Brute force is O(n^2). Storing seen values in a Map gives O(n) time and O(n) space.

PROBLEM 7: Character frequency

Count how many times each character appears in a string.

Solution

```
const freq = s => [...s].reduce((m, ch) => {
  m[ch] = (m[ch] || 0) + 1;
  return m;
}, {});
freq("banana");    // { b: 1, a: 3, n: 2 }
```

PROBLEM 8: Flatten a nested array

Flatten `[1,[2,[3,[4]]]]` into `[1,2,3,4]` without using flat().

Solution

```
function flatten(arr) {
  return arr.reduce((out, x) => out.concat(Array.isArray(x) ? flatten(x) : x), []);
}
flatten([1, [2, [3, [4]]]]);    // [1,2,3,4]
// built in: [1,[2,[3,[4]]]].flat(Infinity)
```

Explanation: Recursion: if an element is an array, flatten it first, otherwise keep it.

PROBLEM 9: Debounce

Implement `debounce(fn, delay)` so that fn runs only after the user stops typing for delay ms.

Solution

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
const onType = debounce(q => console.log("search", q), 300);
```

Explanation: The closure keeps `timer` between calls. Every new call cancels the previous timer. Used in search boxes.

PROBLEM 10: Predict the output (event loop)

What is printed?

Solution

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Output: A  D  C  B
```

Explanation: A and D are synchronous. Then the microtask queue (promise) runs: C. Finally the timer callback runs: B.

PROBLEM 11: Predict the output (var vs let)

What do these two loops print?

Solution

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);   // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j), 0);   // 0 1 2
```

Explanation: var is function scoped, so all callbacks share one i, which is 3 when they finally run. let creates a new binding per iteration.

PROBLEM 12: Fetch and render users

Fetch users from the API and show each name and email as a list item. Show a loading and an error message.

Solution

```
const out = document.querySelector("#out");
async function show() {
  out.textContent = "Loading...";
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("Server error " + res.status);
    const users = await res.json();
    out.innerHTML = "";
    users.forEach(u => {
      const li = document.createElement("li");
      li.textContent = `${u.name} - ${u.email}`;
      out.appendChild(li);
    });
  } catch (e) { out.textContent = "Could not load: " + e.message; }
}
show();
```

PROBLEM 13: Group and count with reduce

Given students with a grade, return an object counting students per grade.

Solution

```
const students = [{n:"A", g:"B"}, {n:"C", g:"A"}, {n:"D", g:"B"}];
const byGrade = students.reduce((acc, s) => ({ ...acc, [s.g]: (acc[s.g] || 0) + 1 }), {});
// { B: 2, A: 1 }
```

PROBLEM 14: Remove duplicates and sort

Given `[5,3,5,1,3,9]` return unique values in ascending order.

Solution

```
const result = [...new Set([5, 3, 5, 1, 3, 9])].sort((a, b) => a - b);   // [1,3,5,9]
```

Explanation: A Set stores only unique values. The compare function is needed because the default sort is alphabetical.

5. React

React is a library for building user interfaces from small reusable **components**. The UI is a function of **state**: when state changes, React re-renders the affected components. React updates the real DOM efficiently using a **virtual DOM** diffing process.

5.1 Project setup

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev          # http://localhost:5173
npm install axios react-router-dom
```

5.2 Components, JSX and props

```
// src/StudentCard.jsx
export default function StudentCard({ name, course, marks = 0 }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Course: {course}</p>
      <p>{marks >= 40 ? "Pass" : "Fail"}</p>     {/* conditional rendering */}
    </div>
  );
}

// App.jsx
import StudentCard from "./StudentCard";
export default function App() {
  const students = [{ id: 1, name: "Asha", course: "Java", marks: 88 },
                    { id: 2, name: "Ravi", course: "Web", marks: 35 }];
  return (
    <div>
      {students.map(s => <StudentCard key={s.id} {...s} />)}   {/* key = stable unique id */}
    </div>
  );
}
```

- **Props** are read-only inputs passed from parent to child. **State** is data owned by the component that can change.
- JSX rules: one root element, `className` instead of `class`, close every tag, JavaScript expressions go inside `{ }`.
- Keys must be stable and unique (an id, not the array index when the list can reorder).

5.3 useState

```
import { useState } from "react";
export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>   {/* functional update */}
      <button onClick={() => setCount(0)}>Reset</button>
    </>
  );
}
```

Never mutate state directly. For arrays and objects always create a new copy: `setItems([...items, newItem])`, `setUser({ ...user, name: 'X' })`.

5.4 useEffect

Runs side effects (fetching data, timers, subscriptions) *after* render. The dependency array controls when it re-runs: `[]` once on mount, `[id]` when id changes, none = every render.

```
useEffect(() => {
  const timer = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(timer);       // cleanup on unmount / before next run
}, []);
```

5.5 Fetching data with loading and error handling

```
import { useEffect, useState } from "react";
import axios from "axios";

export default function StudentList() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    axios.get("http://localhost:8080/api/students")
      .then(res => setStudents(res.data))
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;
  return <ul>{students.map(s => <li key={s.id}>{s.name} ({s.email})</li>)}</ul>;
}
```

5.6 Controlled forms

```
function LoginForm() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});

  const handleChange = e => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = e => {
    e.preventDefault();                       // stop page reload
    const errs = {};
    if (!/^\S+@\S+\.\S+$/.test(form.email)) errs.email = "Invalid email";
    if (form.password.length < 8) errs.password = "Min 8 characters";
    setErrors(errs);
    if (Object.keys(errs).length === 0) console.log("submit", form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={form.email} onChange={handleChange} />
      {errors.email && <small>{errors.email}</small>}
      <input name="password" type="password" value={form.password} onChange={handleChange} />
      {errors.password && <small>{errors.password}</small>}
      <button type="submit">Login</button>
    </form>
  );
}
```

5.7 Lifting state, Context and custom hooks

```
// ThemeContext.jsx - share data without passing props through every level
import { createContext, useContext, useState } from "react";
const ThemeContext = createContext();
export const useTheme = () => useContext(ThemeContext);
export function ThemeProvider({ children }) {
  const [dark, setDark] = useState(false);
  return <ThemeContext.Provider value={{ dark, toggle: () => setDark(d => !d) }}>
           {children}
         </ThemeContext.Provider>;
}

// useFetch.js - reusable logic
export function useFetch(url) {
  const [data, setData] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    const ctrl = new AbortController();
    setLoading(true);
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error("HTTP " + r.status); return r.json(); })
      .then(setData)
      .catch(e => { if (e.name !== "AbortError") setError(e.message); })
      .finally(() => setLoading(false));
    return () => ctrl.abort();
  }, [url]);
  return { data, loading, error };
}
```

5.8 React Router

```
import { BrowserRouter, Routes, Route, Link, useParams, Navigate } from "react-router-dom";

function StudentDetail() {
  const { id } = useParams();                  // /students/7 -> id = "7"
  return <h2>Student #{id}</h2>;
}
export default function App() {
  const loggedIn = !!localStorage.getItem("token");
  return (
    <BrowserRouter>
      <nav><Link to="/">Home</Link> | <Link to="/students">Students</Link></nav>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/students" element={loggedIn ? <StudentList /> : <Navigate to="/login" />} />
        <Route path="/students/:id" element={<StudentDetail />} />
        <Route path="*" element={<h1>404 Not Found</h1>} />
      </Routes>
    </BrowserRouter>
  );
}
```

5.9 Practice problems

PROBLEM 15: Todo app with add, toggle, delete

Build a todo list: add items, click to mark done, and delete items.

Solution

```
import { useState } from "react";
export default function Todo() {
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([]);

  const add = () => {
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text, done: false }]);
    setText("");
  };
  const toggle = id => setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t));
  const remove = id => setTodos(todos.filter(t => t.id !== id));

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button onClick={add}>Add</button>
      <ul>
        {todos.map(t => (
          <li key={t.id}>
            <span onClick={() => toggle(t.id)}
                  style={{ textDecoration: t.done ? "line-through" : "none", cursor: "pointer" }}>
              {t.text}
            </span>
            <button onClick={() => remove(t.id)}>x</button>
          </li>
        ))}
      </ul>
```

```
    </div>
  );
}
```

Explanation: Each update creates a new array (map/filter/spread). Never use `todos.push()` on state.

PROBLEM 16: Live search filter

Show a list of students and filter it as the user types (case-insensitive).

Solution

```
function SearchList({ students }) {
  const [q, setQ] = useState("");
  const filtered = students.filter(s => s.name.toLowerCase().includes(q.toLowerCase()));
  return (
    <>
      <input placeholder="Search..." value={q} onChange={e => setQ(e.target.value)} />
      <ul>{filtered.map(s => <li key={s.id}>{s.name}</li>)}</ul>
      {filtered.length === 0 && <p>No results</p>}
    </>
  );
}
```

Explanation: The filtered list is *derived* from state during render; you do not need a separate state for it.

PROBLEM 17: Infinite re-render bug

Why does this component freeze the browser and how do you fix it?

Solution

```
// BUG: setState during render triggers another render forever
function Bad() {
  const [n, setN] = useState(0);
  setN(n + 1);
  return <p>{n}</p>;
}
// BUG 2: effect with no dependency array that sets state
useEffect(() => { setData(compute()); });          // runs after every render -> loop

// FIX: update state only in event handlers or in effects with correct dependencies
useEffect(() => { setData(compute()); }, []);      // once
```

PROBLEM 18: Parent-child communication

A child `<AddForm />` must send a new student to the parent that owns the list.

Solution

```
function Parent() {
  const [students, setStudents] = useState([]);
  const addStudent = s => setStudents(prev => [...prev, s]);
  return (<><AddForm onAdd={addStudent} /><List items={students} /></>);
}
function AddForm({ onAdd }) {
  const [name, setName] = useState("");
  return (
    <form onSubmit={e => { e.preventDefault(); onAdd({ id: Date.now(), name }); setName(""); }}>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button>Add</button>
    </form>
  );
}
```

Explanation: Data flows down through props; events flow up through callback props. This is called **lifting state up**.

PROBLEM 19: Axios with JWT interceptor

Send the token automatically on every request and redirect to login on a 401 response.

Solution

```
import axios from "axios";
const api = axios.create({ baseURL: "http://localhost:8080/api" });

api.interceptors.request.use(cfg => {
  const token = localStorage.getItem("token");
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);
export default api;      // usage: api.get("/students")
```

6. Java Essentials for Backend

You already know Java; this chapter revises exactly what Spring Boot code uses every day.

6.1 OOP pillars

Pillar	Meaning	Example
Encapsulation	Hide fields, expose via methods	private fields + getters/setters
Inheritance	Reuse via extends	class Student extends Person
Polymorphism	One interface, many behaviours	PaymentService ref holds UPI or Card impl
Abstraction	Hide complexity behind interfaces	interface StudentService

Interface vs abstract class: interface = contract (a class can implement many); abstract class = partial implementation with state (a class extends only one). Spring code relies on interfaces and **dependency injection**: a class receives collaborators instead of creating them with `new`.

6.2 Collections cheat sheet

Type	Ordered?	Duplicates?	Use when
ArrayList	Insertion order	Yes	Fast random access, general list
LinkedList	Insertion order	Yes	Frequent insert/delete at ends
HashSet	No	No	Uniqueness check, O(1)
LinkedHashSet	Insertion order	No	Unique + keep order
TreeSet	Sorted	No	Sorted unique values
HashMap	No	Keys unique	Key -> value lookup, O(1)
LinkedHashMap	Insertion order	Keys unique	Map that remembers order
TreeMap	Sorted by key	Keys unique	Sorted map

6.3 Streams and Optional

```
record Employee(String name, String dept, double salary) {}      // Java 16+

List<Employee> emps = List.of(new Employee("A", "IT", 50000), new Employee("B", "HR", 40000),
                              new Employee("C", "IT", 70000));

List<String> itNames = emps.stream()
        .filter(e -> e.dept().equals("IT"))
        .map(Employee::name)
        .sorted()
        .toList();                                          // [A, C]

double total = emps.stream().mapToDouble(Employee::salary).sum();

Map<String, Double> avgByDept = emps.stream()
        .collect(Collectors.groupingBy(Employee::dept,
              Collectors.averagingDouble(Employee::salary)));

Optional<Employee> top = emps.stream().max(Comparator.comparingDouble(Employee::salary));
String name = top.map(Employee::name).orElse("none");    // avoid get() without isPresent()
```

6.4 Exceptions

- **Checked** exceptions (IOException, SQLException) must be caught or declared; **unchecked** (RuntimeException, NullPointerException) need not be.
- Spring convention: throw your own unchecked exceptions and handle them centrally in `@RestControllerAdvice`.

- Use try-with-resources for anything that must be closed.

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}
```

6.5 Practice problems

PROBLEM 20: Word frequency

Count the frequency of each word in a sentence (case-insensitive).

Solution

```
static Map<String, Long> wordFreq(String text) {
    return Arrays.stream(text.toLowerCase().split("\\s+"))
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
                  Collectors.counting()));
}
// wordFreq("Java is fun and java is fast") -> {java=2, is=2, fun=1, and=1, fast=1}
```

PROBLEM 21: First non-repeating character

Return the first character that appears only once, or '_' if none.

Solution

```
static char firstUnique(String s) {
    Map<Character, Integer> count = new LinkedHashMap<>();
    for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);
    for (var e : count.entrySet()) if (e.getValue() == 1) return e.getKey();
    return '_';
}
// firstUnique("swiss") -> 'w'
```

Explanation: LinkedHashMap keeps insertion order, so the first entry with count 1 is the answer. O(n).

PROBLEM 22: Find duplicates in a list

Return all duplicate elements of a `List<Integer>`.

Solution

```
static Set<Integer> duplicates(List<Integer> list) {
    Set<Integer> seen = new HashSet<>(), dups = new LinkedHashSet<>();
    for (int n : list) if (!seen.add(n)) dups.add(n);       // add() returns false if present
    return dups;
}
// duplicates(List.of(1,2,3,2,4,1)) -> [2, 1]
```

PROBLEM 23: Second highest salary using streams

From a list of employees return the second highest distinct salary.

Solution

```
Optional<Double> second = emps.stream()
        .map(Employee::salary).distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1).findFirst();
```

PROBLEM 24: Implement a simple custom exception flow

Write a service method that throws when a student is not found, and show how it is used.

Solution

```
public Student findById(Long id) {
    return repo.findById(id)
               .orElseThrow(() -> new ResourceNotFoundException("Student " + id + " not found"));
}
```

Explanation: Optional.orElseThrow is the idiomatic way; no null checks needed.

7. Spring Boot REST API (Complete Project)

Spring Boot removes configuration work: it provides an embedded Tomcat server, auto-configures beans from the libraries on your classpath, and manages dependency versions. We will build a complete **Student Management** API with validation, pagination, error handling and CORS.

7.1 Create the project

Go to start.spring.io: Maven, Java 17, latest Spring Boot 3.x. Add dependencies: **Spring Web, Spring Data JPA, Validation, MySQL Driver (or H2 for a quick start), Lombok**.

```
src/main/java/com/example/student/
    StudentApplication.java          // @SpringBootApplication main class
    controller/StudentController.java
    service/StudentService.java
    repository/StudentRepository.java
    model/Student.java               // @Entity
    dto/StudentRequest.java, StudentResponse.java
    exception/ResourceNotFoundException.java, DuplicateResourceException.java,
              GlobalExceptionHandler.java
    config/CorsConfig.java
src/main/resources/application.properties
```

7.2 Core concepts

- **IoC / Dependency Injection**: Spring creates objects (*beans*) and injects them where needed. Prefer **constructor injection**.
- Stereotype annotations: `@Component` (generic), `@Service` (business logic), `@Repository` (data access), `@Controller` / `@RestController` (web layer).
- `@RestController` = `@Controller` + `@ResponseBody`: return values are written as JSON.
- `@SpringBootApplication` = `@Configuration + @EnableAutoConfiguration + @ComponentScan`.
- **Layers**: Controller (HTTP) -> Service (business rules, transactions) -> Repository (database). Never put business logic in controllers or return entities directly to clients (use DTOs).

7.3 Configuration (application.properties)

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

ddl-auto: `update` is fine for learning; use `validate` plus Flyway/Liquibase migrations in production. Never commit real passwords; read them from environment variables.

7.4 Entity and repository

```
@Entity
@Table(name = "students")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
```

```
    private String email;

    private int age;
}

public interface StudentRepository extends JpaRepository<Student, Long> {
    boolean existsByEmail(String email);
    Optional<Student> findByEmail(String email);
    Page<Student> findByNameContainingIgnoreCase(String name, Pageable pageable);

    @Query("SELECT s FROM Student s WHERE s.age BETWEEN :min AND :max")
    List<Student> findByAgeRange(@Param("min") int min, @Param("max") int max);
}
```

JpaRepository gives you `save, findById, findAll, deleteById, count, existsById` without writing SQL. Derived query methods such as `findByEmail` are generated from the method name.

7.5 DTOs with validation

```
public record StudentRequest(
        @NotBlank(message = "Name is required") String name,
        @NotBlank @Email(message = "Invalid email") String email,
        @Min(value = 16, message = "Age must be at least 16") @Max(100) int age) {}

public record StudentResponse(Long id, String name, String email, int age) {}
```

7.6 Service layer

```
@Service
public class StudentService {
    private final StudentRepository repo;
    public StudentService(StudentRepository repo) { this.repo = repo; }   // constructor injection

    @Transactional
    public StudentResponse create(StudentRequest r) {
        if (repo.existsByEmail(r.email()))
            throw new DuplicateResourceException("Email already registered: " + r.email());
        Student s = new Student(null, r.name(), r.email(), r.age());
        return toResponse(repo.save(s));
    }

    @Transactional(readOnly = true)
    public Page<StudentResponse> getAll(String name, Pageable pageable) {
        Page<Student> page = (name == null || name.isBlank())
                ? repo.findAll(pageable)
                : repo.findByNameContainingIgnoreCase(name, pageable);
        return page.map(this::toResponse);
    }

    public StudentResponse getById(Long id) { return toResponse(find(id)); }

    @Transactional
    public StudentResponse update(Long id, StudentRequest r) {
        Student s = find(id);
        s.setName(r.name()); s.setEmail(r.email()); s.setAge(r.age());
        return toResponse(repo.save(s));
    }

    @Transactional
    public void delete(Long id) { repo.delete(find(id)); }

    private Student find(Long id) {
        return repo.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Student " + id + " not found"));
    }
    private StudentResponse toResponse(Student s) {
        return new StudentResponse(s.getId(), s.getName(), s.getEmail(), s.getAge());
    }
}
```

7.7 Controller

```
@RestController
@RequestMapping("/api/students")
```

```
public class StudentController {
    private final StudentService service;
    public StudentController(StudentService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<StudentResponse> create(@Valid @RequestBody StudentRequest req) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.create(req));
    }

    @GetMapping    // GET /api/students?name=as&page=0&size=10&sort=name,asc
    public Page<StudentResponse> list(@RequestParam(required = false) String name, Pageable
          pageable) {
        return service.getAll(name, pageable);
    }

    @GetMapping("/{id}")
    public StudentResponse get(@PathVariable Long id) { return service.getById(id); }

    @PutMapping("/{id}")
    public StudentResponse update(@PathVariable Long id, @Valid @RequestBody StudentRequest req) {
        return service.update(id, req);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

7.8 Global exception handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> notFound(ResourceNotFoundException ex) {
        return body(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<Map<String, Object>> conflict(DuplicateResourceException ex) {
        return body(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> invalid(MethodArgumentNotValidException ex) {
        Map<String, String> fields = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(f -> fields.put(f.getField(), f.getDefaultMessage()));
        Map<String, Object> res = new LinkedHashMap<>();
        res.put("status", 400); res.put("error", "Validation failed"); res.put("fields", fields);
        return ResponseEntity.badRequest().body(res);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> generic(Exception ex) {
        return body(HttpStatus.INTERNAL_SERVER_ERROR, "Something went wrong");   // do not leak
             details
    }

    private ResponseEntity<Map<String, Object>> body(HttpStatus st, String msg) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("timestamp", Instant.now().toString());
        m.put("status", st.value());
        m.put("error", msg);
        return ResponseEntity.status(st).body(m);
    }
}
```

7.9 CORS (so React on port 5173 can call the API on 8080)

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
```

```
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                .allowedHeaders("*");
    }
}
```

CORS is a **browser** rule: a page from one origin (scheme+host+port) may call another origin only if the server allows it. Postman and curl are not affected, which is why an API can work in Postman but fail in the browser.

7.10 JPA relationships

```
@Entity
public class Department {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
}
@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")              // foreign key column
    private Department department;
}
```

- **Owning side** is the side with `@JoinColumn` (the many side); `mappedBy` marks the inverse side.
- **LAZY** loads related data only when accessed; **EAGER** loads it immediately. Prefer LAZY.
- **N+1 problem**: loading 100 departments then touching each one's employees runs 1 + 100 queries. Fix with `JOIN FETCH` or `@EntityGraph`.
- **LazyInitializationException** happens when you touch a lazy field after the transaction/session closed; map to a DTO inside the service.

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();
```

7.11 Test the API with curl

```
curl -X POST http://localhost:8080/api/students -H "Content-Type: application/json" \
     -d '{"name":"Asha","email":"asha@mail.com","age":21}'
curl http://localhost:8080/api/students?page=0&size=5
curl -X PUT http://localhost:8080/api/students/1 -H "Content-Type: application/json" \
     -d '{"name":"Asha K","email":"asha@mail.com","age":22}'
curl -X DELETE http://localhost:8080/api/students/1
```

PROBLEM 25: Add a search endpoint by age range

Add `GET /api/students/age?min=18&max=25` returning matching students.

Solution

```
// Controller
@GetMapping("/age")
public List<StudentResponse> byAge(@RequestParam int min, @RequestParam int max) {
    return service.byAgeRange(min, max);
}
// Service
public List<StudentResponse> byAgeRange(int min, int max) {
    if (min > max) throw new IllegalArgumentException("min must be <= max");
    return repo.findByAgeRange(min, max).stream().map(this::toResponse).toList();
}
```

Explanation: Map `IllegalArgumentException` to a 400 response in your exception handler.

PROBLEM 26: Why does POST return 415 or 400?

Your POST returns 415 Unsupported Media Type. What are the likely causes?

Solution

```
// 1. Missing header          -> add  Content-Type: application/json
// 2. Body sent as form data  -> send raw JSON, not x-www-form-urlencoded
// 3. Missing @RequestBody    -> public ... create(@RequestBody StudentRequest req)
// 400 causes: malformed JSON, wrong field types ("age":"abc"), failed @Valid rules
```

8. SQL, Databases and JPA

8.1 Core ideas

- **Relational database**: data lives in tables (rows and columns) linked by keys.
- **Primary key**: uniquely identifies a row, never null. **Foreign key**: refers to a primary key in another table. **Unique**: no duplicates but may be null.
- **ACID** transactions: **A**tomicity (all or nothing), **C**onsistency (rules always hold), **I**solation (concurrent transactions do not interfere), **D**urability (committed data survives a crash).
- **Index**: a lookup structure (B-tree) that makes searches on a column fast but slows writes slightly. Index columns used in WHERE, JOIN and ORDER BY.

8.2 Schema and basic statements

```
CREATE TABLE departments (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  salary DECIMAL(10,2) CHECK (salary >= 0),
  dept_id INT,
  manager_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(id),
  FOREIGN KEY (manager_id) REFERENCES employees(id)
);

INSERT INTO departments(name) VALUES ('IT'), ('HR'), ('Sales');
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 1;
DELETE FROM employees WHERE id = 5;

SELECT name, salary FROM employees
WHERE salary > 40000 AND name LIKE 'A%'
ORDER BY salary DESC
LIMIT 5;
```

8.3 Joins

Join	Returns
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows of the left table, plus matches (NULL if none)
RIGHT JOIN	All rows of the right table, plus matches
FULL OUTER JOIN	All rows from both (not in MySQL; emulate with UNION)
SELF JOIN	A table joined to itself (e.g. employee and manager)

WHERE filters rows *before* grouping; **HAVING** filters groups *after* GROUP BY.

8.4 Normalization

- **1NF**: each cell holds one atomic value (no comma separated lists).
- **2NF**: 1NF and every non-key column depends on the *whole* primary key.
- **3NF**: 2NF and no non-key column depends on another non-key column (no transitive dependency).

Example: a table *orders(order_id, customer_name, customer_city, product)* repeats customer data. Move customers into their own table and reference them by `customer_id`.

8.5 Practice problems (use the employees/departments schema)

PROBLEM 27: Second highest salary

Find the second highest distinct salary.

Solution

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- generic Nth highest (N = 2):
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

PROBLEM 28: Find duplicate emails

A users table has duplicate emails. List each duplicated email and how many times it appears.

Solution

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

PROBLEM 29: Employees per department including empty departments

Show every department with its number of employees (0 if none).

Solution

```
SELECT d.name, COUNT(e.id) AS emp_count
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.id
GROUP BY d.id, d.name;
```

Explanation: A LEFT JOIN keeps departments without employees; `COUNT(e.id)` ignores NULLs so they show 0.

PROBLEM 30: Employees earning more than their manager

Return names of employees whose salary is higher than their manager's salary.

Solution

```
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Explanation: Self join: the same table is used twice under different aliases (e = employee, m = manager).

PROBLEM 31: Top 3 salaries in each department

Return the three highest paid employees of every department.

Solution

```
SELECT name, dept_id, salary
FROM (
  SELECT name, dept_id, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk <= 3;
```

Explanation: Window functions rank rows inside each partition without collapsing them like GROUP BY.

PROBLEM 32: Delete duplicate rows keeping the smallest id

Remove duplicate emails from users but keep one row per email.

Solution

```
DELETE u1 FROM users u1
JOIN users u2 ON u1.email = u2.email AND u1.id > u2.id;       -- MySQL

-- PostgreSQL version
DELETE FROM users a USING users b WHERE a.email = b.email AND a.id > b.id;
```

PROBLEM 33: Departments with average salary above 50000

Show department names whose average salary is greater than 50000.

Solution

```
SELECT d.name, AVG(e.salary) AS avg_sal
FROM departments d JOIN employees e ON e.dept_id = d.id
GROUP BY d.id, d.name
HAVING AVG(e.salary) > 50000;
```

Explanation: Aggregate conditions must be in HAVING, not WHERE.

PROBLEM 34: Bank transfer with a transaction

Transfer 500 from account 1 to account 2 so that either both updates happen or neither.

Solution

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1 AND balance >= 500;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;      -- use ROLLBACK; if any step failed

// In Spring: put @Transactional on the service method; any RuntimeException rolls back
      automatically.
```

8.6 SQL injection

```
// VULNERABLE: user input is concatenated into SQL
String sql = "SELECT * FROM users WHERE email = '" + email + "'";     // email = ' OR '1'='1

// SAFE: prepared statement (JPA/Spring Data do this for you)
PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE email = ?");
ps.setString(1, email);
```

9. Authentication and Security

9.1 Key terms

- **Authentication**: who are you? (login). **Authorization**: what are you allowed to do? (roles).
- **Never store plain passwords.** Store a salted slow hash using **BCrypt**.
- **Session auth**: server stores session, browser keeps a cookie. **JWT auth**: server signs a token that contains the user info; the client sends it in the `Authorization: Bearer <token>` header; the server stays stateless.
- A JWT has three Base64 parts: **header.payload.signature**. The payload is *readable by anyone*, so never put secrets in it. The signature proves it was not tampered with.

9.2 JWT login flow

```
1. POST /api/auth/login  {email, password}
2. Server checks the BCrypt hash; if valid, creates a signed JWT (with expiry) and returns it
3. React stores the token and sends  Authorization: Bearer <token>  on every request
4. A server filter validates the signature and expiry, then sets the authenticated user
5. Controllers/roles are checked: @PreAuthorize("hasRole('ADMIN')")
```

9.3 Dependencies (pom.xml)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

9.4 JwtUtil

```
@Component
public class JwtUtil {
    private final SecretKey key;
    private final long expirationMs = 1000L * 60 * 60;           // 1 hour

    public JwtUtil(@Value("${app.jwt.secret}") String secret) {  // at least 32 chars, from env
          var
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }
    public String generateToken(String username, String role) {
        return Jwts.builder()
                .subject(username)
                .claim("role", role)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expirationMs))
                .signWith(key)
                .compact();
    }
    public String extractUsername(String token) {
        return Jwts.parser().verifyWith(key).build()
                   .parseSignedClaims(token).getPayload().getSubject();   // throws JwtException
                         if invalid
    }
}
```

9.5 JWT filter

```
@Component
public class JwtFilter extends OncePerRequestFilter {
    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;
    public JwtFilter(JwtUtil jwtUtil, UserDetailsService uds) {
        this.jwtUtil = jwtUtil; this.userDetailsService = uds;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
          chain)
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                String username = jwtUtil.extractUsername(header.substring(7));
                if (SecurityContextHolder.getContext().getAuthentication() == null) {
                    UserDetails user = userDetailsService.loadUserByUsername(username);
                    var auth = new UsernamePasswordAuthenticationToken(user, null,
                          user.getAuthorities());
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            } catch (JwtException | IllegalArgumentException e) {
                // invalid or expired token: continue unauthenticated, security rules will return
                      401
            }
        }
        chain.doFilter(req, res);
    }
}
```

9.6 Security configuration (Spring Security 6)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    private final JwtFilter jwtFilter;
    public SecurityConfig(JwtFilter jwtFilter) { this.jwtFilter = jwtFilter; }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())                       // OK for stateless token APIs
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
    @Bean AuthenticationManager authManager(AuthenticationConfiguration c) throws Exception {
        return c.getAuthenticationManager();
    }
}
```

9.7 Register and login endpoints

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final AuthenticationManager authManager;
    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final JwtUtil jwtUtil;
    // constructor omitted for brevity

    @PostMapping("/register")
    public ResponseEntity<Void> register(@Valid @RequestBody RegisterRequest r) {
        if (users.existsByEmail(r.email())) throw new DuplicateResourceException("Email taken");
```

```
        users.save(new User(null, r.email(), encoder.encode(r.password()), "USER"));   // hash!
        return ResponseEntity.status(HttpStatus.CREATED).build();
    }

    @PostMapping("/login")
    public Map<String, String> login(@RequestBody LoginRequest r) {
        authManager.authenticate(new UsernamePasswordAuthenticationToken(r.email(),
              r.password()));
        User u = users.findByEmail(r.email()).orElseThrow();
        return Map.of("token", jwtUtil.generateToken(u.getEmail(), u.getRole()));
    }
}
```

You also need a `UserDetailsService` bean that loads your User from the database and returns a Spring Security `UserDetails` with the role as `ROLE_USER` / `ROLE_ADMIN`.

9.8 Common web vulnerabilities

Attack	What it is	Defence
SQL injection	Input changes SQL meaning	Prepared statements / JPA, validate input
XSS	Attacker script runs in victim browser	Escape output; React escapes by default; avoid dangerouslySetInnerHTML
CSRF	Another site forces your browser to act	CSRF tokens / SameSite cookies (not needed for header-token APIs)
Broken auth	Weak passwords/tokens	BCrypt, short token expiry, HTTPS, rate limit login
Sensitive data exposure	Returning password hash/secrets	Use DTOs; never log tokens

- Token storage: **localStorage** is easy but readable by XSS; an **httpOnly, Secure, SameSite cookie** is safer against XSS. Know the trade-off for interviews.
- Always use HTTPS in production. Keep secrets in environment variables, not in Git.

PROBLEM 35: Protect an endpoint by role

Only ADMIN can delete a student; any logged in user can read.

Solution

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> delete(@PathVariable Long id) { ... }

// 401 = no/invalid token, 403 = valid token but wrong role
```

Explanation: Requires `@EnableMethodSecurity` on the config class. The role stored as ADMIN becomes authority ROLE_ADMIN.

10. Testing

Tests prove your code works and protect you when you change it. Interviewers like candidates who test.

10.1 Test pyramid

- **Unit tests** (many, fast): one class in isolation with mocked dependencies (JUnit 5 + Mockito).
- **Integration/slice tests**: web layer with `@WebMvcTest`, database layer with `@DataJpaTest`.
- **End-to-end tests** (few, slow): the whole app through the UI (Cypress/Playwright).

10.2 Service unit test with Mockito

```
@ExtendWith(MockitoExtension.class)
class StudentServiceTest {
    @Mock StudentRepository repo;
    @InjectMocks StudentService service;

    @Test
    void create_savesStudent_whenEmailIsNew() {
        StudentRequest req = new StudentRequest("Asha", "asha@mail.com", 21);
        when(repo.existsByEmail("asha@mail.com")).thenReturn(false);
        when(repo.save(any(Student.class))).thenAnswer(inv -> {
            Student s = inv.getArgument(0); s.setId(1L); return s;
        });

        StudentResponse res = service.create(req);

        assertEquals(1L, res.id());
        assertEquals("Asha", res.name());
        verify(repo).save(any(Student.class));
    }

    @Test
    void create_throws_whenEmailExists() {
        when(repo.existsByEmail("a@b.com")).thenReturn(true);
        assertThrows(DuplicateResourceException.class,
                     () -> service.create(new StudentRequest("A", "a@b.com", 20)));
        verify(repo, never()).save(any());
    }

    @Test
    void getById_throws_whenMissing() {
        when(repo.findById(99L)).thenReturn(Optional.empty());
        assertThrows(ResourceNotFoundException.class, () -> service.getById(99L));
    }
}
```

10.3 Controller test with MockMvc

```
@WebMvcTest(StudentController.class)
class StudentControllerTest {
    @Autowired MockMvc mvc;
    @MockBean StudentService service;           // @MockitoBean in Spring Boot 3.4+

    @Test
    void get_returns404_whenMissing() throws Exception {
        when(service.getById(5L)).thenThrow(new ResourceNotFoundException("Student 5 not found"));
        mvc.perform(get("/api/students/5"))
           .andExpect(status().isNotFound())
           .andExpect(jsonPath("$.error").value("Student 5 not found"));
    }

    @Test
    void post_returns400_whenInvalid() throws Exception {
        String bad = "{\"name\":\"\",\"email\":\"x\",\"age\":5}";
        mvc.perform(post("/api/students").contentType(MediaType.APPLICATION_JSON).content(bad))
           .andExpect(status().isBadRequest());
    }
}
```

10.4 React component test (Vitest + Testing Library)

```
import { render, screen, fireEvent } from "@testing-library/react";
import Counter from "./Counter";

test("increments the counter", () => {
  render(<Counter />);
  fireEvent.click(screen.getByText("+1"));
  expect(screen.getByText("Count: 1")).toBeInTheDocument();
});
```

PROBLEM 36: Write a test for a palindrome function

Write JUnit tests for a Java method `isPalindrome(String)` covering normal, edge and null cases.

Solution

```
@ParameterizedTest
@ValueSource(strings = {"madam", "A man a plan a canal Panama", "a", ""})
void palindromes(String s) { assertTrue(Util.isPalindrome(s)); }

@Test void notPalindrome() { assertFalse(Util.isPalindrome("hello")); }
@Test void nullThrows() { assertThrows(IllegalArgumentException.class, () ->
      Util.isPalindrome(null)); }
```

Explanation: Good tests cover typical input, boundaries (empty, single character) and invalid input.

11. Git and GitHub

11.1 Daily commands

Command	What it does
`git init / git clone <url>`	Start a repo / copy a remote repo
`git status`	See changed and staged files
`git add . / git add file`	Stage changes
`git commit -m "message"`	Save a snapshot locally
`git log --oneline --graph`	View history
`git branch / git switch -c feature/login`	List / create and switch branch
`git merge feature/login`	Merge a branch into the current one
`git pull / git push`	Download / upload commits
`git restore file`	Discard unstaged changes
`git stash / git stash pop`	Temporarily shelve changes
`git revert <hash>`	Undo a commit safely by making a new commit

11.2 First push to GitHub

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<username>/<repo>.git
git push -u origin main
```

11.3 Feature branch workflow

```
git switch main && git pull
git switch -c feature/student-api
# ...work, then...
git add . && git commit -m "feat: add student CRUD endpoints"
git push -u origin feature/student-api
# open a Pull Request on GitHub, get review, merge, delete branch
```

11.4 .gitignore and good habits

```
# Java / Maven
target/
.idea/
*.iml
# Node / React
node_modules/
dist/
.env
# secrets
application-local.properties
```

- Write meaningful messages: `fix: handle null email`, `feat: add JWT login`.
- Commit small and often. Never commit passwords, tokens or `.env` files.
- Merge conflict: open the file, keep the right lines, remove the `<<<<<<< ======= >>>>>>>` markers, then `git add` and commit.
- If your images do not appear on GitHub Pages: check exact file name case, relative paths and that the images were actually committed (`git status` must not show them as untracked).

12. Docker and Deployment

12.1 Environment variables

Configuration that changes between laptop and production (database URL, passwords, JWT secret) must come from **environment variables**, not source code.

```
# application.properties
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
app.jwt.secret=${JWT_SECRET}

# React (Vite): variables must start with VITE_
VITE_API_URL=https://my-api.onrender.com/api
// usage: axios.create({ baseURL: import.meta.env.VITE_API_URL })
```

12.2 Build the artifacts

```
# Backend -> single runnable jar
mvn clean package -DskipTests
java -jar target/student-0.0.1-SNAPSHOT.jar

# Frontend -> static files in dist/
npm run build
```

12.3 Dockerfile for Spring Boot (multi-stage)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

12.4 docker-compose with a database

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: studentdb
    volumes: [dbdata:/var/lib/mysql]
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      DB_URL: jdbc:mysql://db:3306/studentdb
      DB_USER: root
      DB_PASSWORD: rootpass
      JWT_SECRET: change-this-to-a-long-random-secret-value
    depends_on: [db]
volumes:
  dbdata:
```

Run with `docker compose up --build`. Inside Docker the database host is the service name `db`, not `localhost`.

12.5 Where to deploy (free tiers change, check current plans)

Part	Options
React frontend	Vercel, Netlify, GitHub Pages

Part	Options
Spring Boot backend	Render, Railway, Fly.io, AWS Elastic Beanstalk
Database	Managed MySQL/PostgreSQL (Railway, Neon, Supabase, AWS RDS)

- After deploying, update CORS to allow your real frontend URL and set the frontend API URL variable.
- Add a GitHub Actions workflow to run tests automatically on every push (CI).

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - run: mvn -B test
```

13. Capstone Project Blueprint

Recruiters trust deployed projects with a README. Build **one polished project** instead of many half-finished ones. Example: **Student / Task / Job Tracker**.

13.1 Features checklist

- User registration and login with JWT; roles USER and ADMIN.
- CRUD for the main entity with validation, search, sorting and pagination.
- Relationship example (e.g. Course has many Students or Project has many Tasks).
- React UI: login page, protected routes, list with search, create/edit form, loading and error states.
- Global exception handling, DTOs, at least 5 unit tests and 2 controller tests.
- Docker setup, environment variables, deployed live URL, README with screenshots and API list.

13.2 Sample API list

Method	Endpoint	Access
POST	/api/auth/register	Public
POST	/api/auth/login	Public
GET	/api/tasks?status=OPEN&page;=0&size;=10	User
POST	/api/tasks	User
PUT	/api/tasks/{id}	Owner
DELETE	/api/tasks/{id}	Admin
GET	/api/admin/users	Admin

13.3 README template

```
# Project Name
One-line description. Live demo: <url>

## Features
## Tech stack      (React, Spring Boot, MySQL, JWT, Docker)
## Screenshots
## Run locally    (git clone, set env vars, mvn spring-boot:run, npm run dev)
## API endpoints
## What I learned
```

14. Common Errors and Fixes

Error	Likely cause	Fix
CORS error in browser console	Backend does not allow the frontend origin	Add CorsConfig / allowedOrigins with the exact origin (no trailing slash)
404 on every endpoint	Wrong path or controller not scanned	Main class must be in a parent package of controllers; check @RequestMapping
Port 8080 already in use	Another process uses it	Stop it or set server.port=8081
Failed to configure a DataSource	Missing DB URL/driver	Add the driver dependency and spring.datasource.* properties
Access denied for user (MySQL)	Wrong credentials	Check username/password; create the schema
415 Unsupported Media Type	Missing JSON Content-Type	Send Content-Type: application/json and use @RequestBody
400 Bad Request on POST	Malformed JSON or failed @Valid	Read the response body; fix field names/types
LazyInitializationException	Lazy field accessed outside transaction	Map to DTO in service or use JOIN FETCH
StackOverflowError on JSON	Bidirectional entity relation	Return DTOs (or use @JsonIgnore / @JsonManagedReference)
No qualifying bean of type	Class not a bean / not scanned	Add @Service/@Component, check package structure
401 after login	Token not sent or expired	Send Authorization: Bearer ; check expiry and secret
React: Cannot read properties of undefined	Data not loaded yet	Use optional chaining and loading checks: user?.name
React: each child needs a key	Missing key in map()	Add key={item.id}
Image not showing	Wrong path/case or not committed	Use relative paths, match case exactly, commit the images
git push rejected	Remote has new commits	git pull --rebase, resolve conflicts, push again

15. Interview Questions and Answers

Q1. What happens when you enter a URL in the browser?

DNS lookup -> TCP/TLS connection -> HTTP request -> server processes and responds -> browser builds DOM/CSSOM, runs JS and renders the page.

Q2. == vs === in JavaScript?

== converts types before comparing; === compares value and type with no conversion. Use ===.

Q3. var vs let vs const?

var is function scoped and hoisted; let and const are block scoped; const cannot be reassigned (the object it points to can still be mutated).

Q4. What is a closure?

A function that keeps access to variables of its outer scope after the outer function returned. Used for private state, counters, debounce.

Q5. Explain the event loop.

The call stack runs synchronous code. Async work is handled by browser APIs and its callbacks are queued. When the stack is empty the loop runs all microtasks (promises) first, then one macrotask (timers, events).

Q6. What is the virtual DOM?

An in-memory representation of the UI. React compares the new tree with the previous one (diffing) and updates only the changed real DOM nodes.

Q7. props vs state?

Props are inputs from the parent and read-only; state is local data owned by a component that triggers a re-render when updated.

Q8. Why are keys needed in lists?

Keys let React identify which items changed, were added or removed, so it can reuse DOM nodes correctly.

Q9. How does the useEffect dependency array work?

[] runs once after the first render; [a, b] runs after renders where a or b changed; no array runs after every render. The returned function is cleanup.

Q10. What is dependency injection / IoC in Spring?

Spring creates and wires objects (beans) so classes do not instantiate their dependencies. It improves loose coupling and testability. Prefer constructor injection.

Q11. @Component vs @Service vs @Repository?

All register a bean. @Service marks business logic, @Repository marks data access and translates persistence exceptions, @Component is generic.

Q12. @Controller vs @RestController?

@RestController = @Controller + @ResponseBody; it returns data (JSON) instead of view names.

Q13. How does Spring Boot auto-configuration work?

@EnableAutoConfiguration scans the classpath and registers beans conditionally (@ConditionalOnClass etc.), e.g. a DataSource if a JDBC driver is present. You can override any auto-configured bean.

Q14. PUT vs PATCH? What is idempotency?

PUT replaces the whole resource, PATCH changes part of it. An idempotent request has the same effect when repeated (GET, PUT, DELETE); POST is not idempotent.

Q15. LAZY vs EAGER loading and the N+1 problem?

LAZY loads associations on first access, EAGER immediately. N+1: one query for the parent list plus one per child. Fix with JOIN FETCH, @EntityGraph or batch fetching.

Q16. JWT vs session authentication?

Sessions keep state on the server; JWT is a signed self-contained token so the server is stateless and scales easily, but tokens cannot be revoked easily before expiry.

Q17. How do you store passwords?

Never plain text. Hash with a slow, salted algorithm such as BCrypt (or Argon2) and compare hashes at login.

Q18. What is CORS and how do you fix it?

A browser security policy blocking cross-origin calls unless the server sends allow headers. Configure allowed origins/methods/headers on the backend.

Q19. WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q20. Primary key vs unique key vs foreign key?

Primary: unique and not null, one per table. Unique: no duplicates, can be null. Foreign: references another table's key to enforce relationships.

Q21. What are ACID properties?

Atomicity, Consistency, Isolation, Durability: guarantees that make transactions reliable.

Q22. How do you prevent SQL injection?

Use prepared statements/parameterized queries or ORM, validate input, use least-privilege DB accounts.

Q23. How does HashMap work internally?

Key hash selects a bucket; collisions are stored in a linked list/tree in that bucket. Average O(1) get/put. Needs correct equals() and hashCode().

Q24. Checked vs unchecked exceptions?

Checked must be handled or declared at compile time (IOException); unchecked (RuntimeException) need not be.

Q25. Interface vs abstract class?

Interface defines a contract (multiple inheritance of type); abstract class can hold state and partial implementation but a class extends only one.

16. 8-Week Study Plan

Week	Focus	Deliverable
1	HTML, CSS (box model, flexbox, grid, responsive)	Responsive portfolio page on GitHub Pages
2	JavaScript core, DOM, async/fetch	Todo app and weather/user-list app using a public API
3	React: components, state, effects, forms, router	React todo or notes app calling a public API
4	Java recap + SQL (joins, group by, window functions)	Solve 30 SQL and 30 Java problems
5	Spring Boot: REST, JPA, validation, exceptions	Student CRUD API tested in Postman
6	Spring Security + JWT, connect React to API	Login/register + protected pages
7	Testing, Docker, deploy, CI	Live deployed app, 10+ tests, README
8	DSA and mock interviews, resume polish	20 DSA problems, 2 mock interviews, resume ready

Daily routine suggestion

- 2 hours learning a concept + 2 hours building something with it (build more than you watch).
- 30-60 minutes of DSA every day (arrays, strings, hashmaps, two pointers, recursion, trees).
- Push to GitHub every day; a green contribution graph shows consistency.
- When stuck, read the error message fully, search it, then ask. Keep an error notebook.

Final advice: projects + clear explanation of your code + basic DSA is the formula for freshers. Be ready to explain every line of your own project.